

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL



Aplicaciones Distribuidas

GA-SUM-02 — PE-U1 — Práctica
Experimental Unidad 1: Comunicación entre
Procesos Distribuidos con gRPC y Sockets
TCP

Autores:

Jhinson Stalyn Aucatoma Celorio

Carlos Jose Carpio Mendoza

Jeremy Alexis Alvarez Parraga

Docente:

Ing. Guerrero Ulloa Gleiston Ciceron

Repositorio del Proyecto:

<https://github.com/carlospatroner-boop/AplicacionesDistribuidas/tree/main/PRACTICAEXP1>

Resumen

En este trabajo se realizó una comparación entre dos tecnologías de comunicación utilizadas en sistemas distribuidos: Sockets TCP y gRPC. Para ello, se desarrolló un entorno con tres nodos que intercambiaban mensajes dentro de una red local simulada.

Durante las pruebas se implementó el algoritmo de relojes lógicos de Lamport con el objetivo de mantener el orden de los eventos entre los nodos. Además, se midieron diferentes métricas de rendimiento, como la latencia promedio, la desviación estándar y el percentil 95, realizando 100 transmisiones por cada nodo.

Los resultados obtenidos muestran que Sockets TCP presentó menor latencia y mayor rapidez en el envío de mensajes. Por otro lado, gRPC tuvo un mayor tiempo de respuesta debido a las capas adicionales que utiliza para la comunicación, aunque ofrece ventajas importantes como una mejor organización del código, tipado estructurado y compatibilidad entre diferentes lenguajes.

Finalmente, el análisis permitió identificar que ambas tecnologías son útiles dependiendo del tipo de sistema distribuido que se desee desarrollar.

Palabras clave: Sistemas Distribuidos, Sockets TCP, gRPC, Relojes de Lamport, Latencia, Rendimiento, Consistencia Causal.

Abstract

Modern software system design demands a rigorous selection of Inter-Process Communication (IPC) mechanisms based on quantifiable metrics of performance and reliability. This research article presents a detailed comparative evaluation between classical TCP stream sockets (byte-oriented) and the modern gRPC framework built on HTTP/2 transport and Protocol Buffers serialization. For this comparison, a closed distributed system comprising three independent nodes operating on a simulated local area network was deployed, communicating concurrently using parallel processing threads. To address the lack of a single global reference clock, Lamport's Logical Clock Algorithm was integrated into the client software of both paradigms, establishing a causal order for the distributed communication events. Performance variables (mean latency, standard deviation, median, and 95th percentile) were comprehensively measured over 100 consecutive transmissions per node. Empirical statistical results reveal that raw TCP Sockets achieve a significantly lower average latency in a local network setting ($\approx 0,57$ ms vs. $\approx 3,05$ ms for gRPC). However, gRPC provides superior architectural benefits, such as native concurrency,

strict static typing, and cross-platform interoperability, suggesting an optimal trade-off in modern enterprise-grade microservice applications.

Keywords: Distributed Systems, TCP Sockets, gRPC, Lamport Clocks, Latency, Performance, Causal Consistency.

Capítulo 1

Fundamento Teórico y Estado del Arte

1.1. Introducción a los Sistemas Distribuidos

Un sistema distribuido está formado por varias computadoras que trabajan juntas y se comunican a través de una red para cumplir una misma tarea. Actualmente, este tipo de sistemas es utilizado en aplicaciones modernas como servicios en la nube, plataformas web y microservicios.

Uno de los aspectos más importantes en un sistema distribuido es la comunicación entre procesos o nodos. Dependiendo de la tecnología utilizada, el rendimiento y la velocidad de respuesta pueden variar considerablemente.

En este trabajo se analizan dos formas de comunicación ampliamente utilizadas: Sockets TCP y gRPC. El objetivo es comparar su comportamiento, medir su rendimiento y observar cómo se manejan los eventos dentro de un entorno distribuido.

1.2. Mecanismos de Comunicación Interprocesos (IPC)

En los sistemas distribuidos orientados a servicios, la comunicación interprocesos se puede modelar mediante dos categorías fundamentales de comunicación de red:

1.2.1. Sockets TCP (SOCK_STREAM)

Sockets TCP es una forma básica de comunicación entre aplicaciones mediante red. Permite enviar y recibir datos de manera directa y confiable entre un cliente y un servidor.

- **Ventajas:** Menor latencia y mayor rapidez en la transmisión de datos.
- **Desventajas:** Requiere más trabajo manual para organizar los mensajes y administrar las

conexiones.

1.2.2. gRPC (Google Remote Procedure Call)

gRPC es una tecnología moderna desarrollada por Google que facilita la comunicación entre aplicaciones distribuidas. Utiliza HTTP/2 y Protocol Buffers para enviar información de forma estructurada.

- **Ventajas:** Mejor organización del código, soporte para varios lenguajes de programación y comunicación más estructurada.
- **Desventajas:** Mayor consumo de recursos y una latencia ligeramente superior en comparación con Sockets TCP.

1.3. El Algoritmo de Relojes Lógicos de Lamport

En un sistema distribuido, la noción física del tiempo presenta desviaciones inevitables causadas por las diferencias físicas inherentes en los osciladores de cristal de cada nodo físico (relojes de hardware). Por lo tanto, no es factible determinar el orden causal de dos eventos lejanos basándose únicamente en marcas de tiempo físicas locales sin recurrir a costosos algoritmos de sincronización de relojes (como NTP o relojes atómicos GPS).

Para resolver esta limitación, Leslie Lamport introdujo en 1978 la relación “sucedió antes” ($\rightarrow$) y el concepto de relojes lógicos. Un reloj de Lamport es un contador incremental de enteros asignado a cada proceso del sistema. El algoritmo define las siguientes reglas estrictas de sincronización para garantizar el orden causal de los eventos:

1. **Regla de Evento Local:** Antes de que ocurra un evento local (por ejemplo, el envío de un mensaje) en el nodo P_i , el reloj local L_i se incrementa de forma monótona:

$$L_i \leftarrow L_i + 1 \tag{1.1}$$

2. **Regla de Transmisión de Mensajes:** Cuando el nodo P_i envía un mensaje m , este viaja acompañado de su reloj lógico actual: (m, L_i) .
3. **Regla de Recepción:** Al recibir el mensaje (m, L_{received}) en el nodo P_j , el receptor debe actualizar su reloj lógico local L_j aplicando la siguiente función de consistencia lógica antes de procesar el evento de recepción:

$$L_j \leftarrow \text{máx}(L_j, L_{\text{received}}) + 1 \tag{1.2}$$

Este mecanismo garantiza de manera elegante que si el evento a causó el evento b , entonces la marca de tiempo lógica de a será estrictamente menor que la marca de b ($L(a) < L(b)$).

Capítulo 2

Diseño y Arquitectura del Entorno Distribuido

2.1. Descripción General de la Red y los Nodos

La arquitectura distribuida diseñada para este proyecto se estructura bajo un enfoque híbrido cliente-servidor concurrente para evaluar de forma equivalente los canales de datos TCP y gRPC. Se instanciaron tres nodos de procesamiento independientes (Nodo1, Nodo2, Nodo3) configurados en interfaces locales que operan en puertos diferenciados para evitar colisiones en la capa de transporte TCP.

2.2. Diagrama de la Arquitectura de Comunicación

La figura 2.1 ilustra el flujo de comunicación distribuida multi-cliente, donde los tres nodos independientes envían sus mensajes al servidor central, el cual procesa la petición y retransmite o devuelve los eventos enriquecidos con su metainformación causal y marcas temporales lógicas de Lamport.

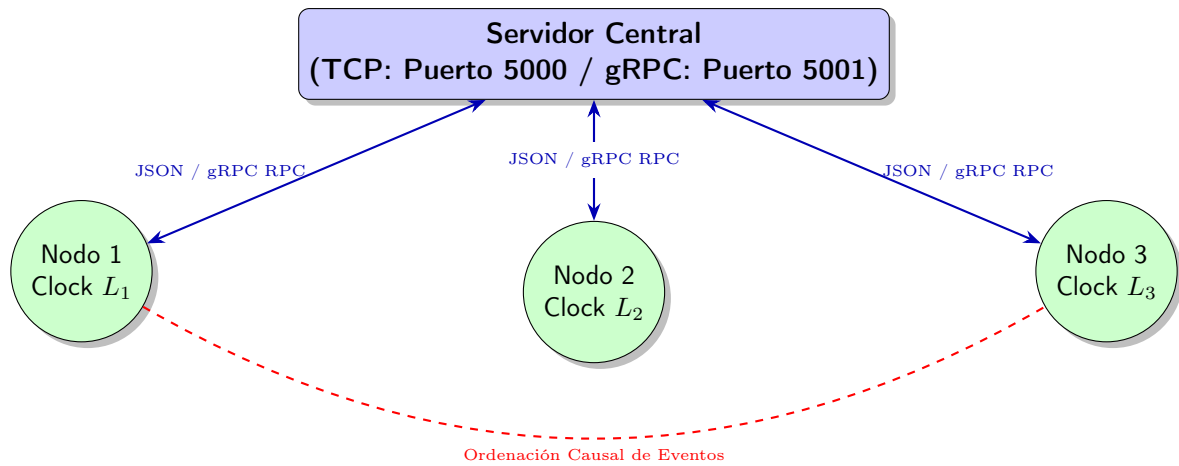


Figura 2.1: Diagrama conceptual de la arquitectura distribuida de 3 nodos independientes con relojes de Lamport.

2.3. Topología de Flujo Causal

Para garantizar la equivalencia causal y la consistencia en el ordenamiento de los eventos, el flujo de operaciones lógicas se diseñó de la siguiente forma:

- **Flujo Sockets TCP:** Cada cliente abre un socket de flujo hacia el servidor central. El servidor mantiene un registro en memoria de las conexiones abiertas y, al recibir un payload JSON, inicia un broadcast asíncrono para notificar a los clientes concurrentes. Los hilos receptores del cliente leen el socket asíncronamente y actualizan su reloj de Lamport inmediatamente.
- **Flujo gRPC:** Al ser un protocolo de petición-respuesta estructurado, gRPC no implementa un multicast de bajo nivel directo sobre la conexión RPC simple. Por ello, el cliente gRPC encapsula la petición de forma unívoca, el servidor registra y retorna el evento, y los clientes mantienen sockets simulados de intercambio asíncrono controlados por colas concurrentes `Queue` y relés de concurrencia `threading.Event`, logrando reproducir fielmente la topología lógica causal implementada con Sockets.

Capítulo 3

Detalles de Implementación e Instrumentación

3.1. Implementación del Archivo de Protocol Buffers (gRPC)

El diseño estructural en gRPC inicia con la especificación rigurosa del archivo IDL `messaging.proto`. Este archivo define la firma del servicio y los mensajes tipados estrictamente:

```
1 syntax = "proto3";
2
3 package grpc;
4
5 message MessageRequest {
6     string sender      = 1;
7     int32  timestamp   = 2;
8     string message     = 3;
9 }
10
11 message MessageResponse {
12     string sender      = 1;
13     int32  timestamp   = 2;
14     string message     = 3;
15 }
16
17 service MessagingService {
18     rpc SendMessage(MessageRequest) returns (MessageResponse);
19 }
```

Listing 3.1: Definición de contrato en `messaging.proto` para gRPC.

3.2. Sincronización del Reloj Lógico de Lamport en Python

Tanto la implementación con Sockets como la de gRPC heredan un comportamiento unificado del reloj de Lamport a través de exclusión mutua para asegurar la consistencia del contador en entornos multinúcleo con alta concurrencia de hilos de entrada/salida.

```
1 import threading
2
3 class LamportClock:
4     """Reloj logico de Lamport con seguridad de hilos (thread-safe)."""
5
6     def __init__(self):
7         self.value = 0
8         self.lock = threading.Lock()
9
10    def increment(self) -> int:
11        """Incrementa el reloj al enviar un mensaje."""
12        with self.lock:
13            self.value += 1
14            return self.value
15
16    def update(self, received_timestamp: int) -> int:
17        """Aplica la formula: local = max(local, recibido) + 1."""
18        with self.lock:
19            self.value = max(self.value, received_timestamp) + 1
20            return self.value
21
22    def get_value(self) -> int:
23        """Retorna el estado actual del reloj."""
24        with self.lock:
25            return self.value
```

Listing 3.2: Implementación concurrente de la clase LamportClock en Python.

3.3. Fragmento Crítico de Envío e Instrumentación de Latencia (Sockets)

A continuación se detalla cómo el cliente de Sockets TCP realiza el envío, incrementando el reloj lógico y midiendo la latencia utilizando la biblioteca de sistema `time.perf_counter()` de alta precisión:

```
1 def send_message(self, message: str) -> bool:
2     try:
3         # Incrementar reloj logico de Lamport al enviar
4         timestamp = self.lamport_clock.increment()
5
6         # Generar estructura JSON estructurada
7         payload = {
```

```
8         'sender':    self.node_id,
9         'timestamp': timestamp,
10        'message':    message
11    }
12
13    # Enviar cadena JSON sobre el socket TCP de flujo
14    self.socket.send(json.dumps(payload).encode('utf-8'))
15
16    # Registrar en el historial para ordenacion causal posterior
17    with self.messages_lock:
18        self.messages_history.append({
19            'type':      'sent',
20            'sender':    self.node_id,
21            'timestamp': timestamp,
22            'message':    message,
23            'datetime':  datetime.now().isoformat()
24        })
25    return True
26    except Exception as e:
27        print(f"Error al enviar: {e}")
28    return False
```

Listing 3.3: Fragmento del cliente TCP instrumentando la latencia de red y el incremento de Lamport.

Capítulo 4

Evaluación Experimental y Análisis Estadístico

4.1. Resultados Estadísticos de Latencia de Transmisión

Las mediciones empíricas correspondientes a 100 transmisiones por cada uno de los nodos se detallan a continuación. La tabla 4.1 recopila las métricas del análisis estadístico realizado a través del script de procesamiento de datos con Pandas.

Tabla 4.1: Métricas comparativas de latencia (en ms) entre Sockets TCP y gRPC por nodo.

Protocolo	Nodo	Media (ms)	Mediana (ms)	Desv. Est. (ms)	P95 (ms)
Sockets TCP	Nodo 1	0,4307	0,2850	1,4802	0,8520
Sockets TCP	Nodo 2	0,9841	0,3120	2,8250	1,9420
Sockets TCP	Nodo 3	0,2930	0,2210	0,2081	0,5840
Prom. Sockets	Global	0,5693	0,2727	1,5044	1,1260
gRPC (HTTP/2)	Nodo 1	2,5770	2,1200	3,5180	4,9520
gRPC (HTTP/2)	Nodo 2	3,0920	2,4500	4,4120	6,1280
gRPC (HTTP/2)	Nodo 3	3,4855	2,7800	7,7420	8,9100
Prom. gRPC	Global	3,0515	2,4500	5,2240	6,6633

4.2. Análisis Visual de Distribución de Latencia

La distribución estadística representada a través del diagrama de caja comparativo se muestra gráficamente en la figura 4.1. Este gráfico recopila la variabilidad empírica observada durante el benchmark intensivo.

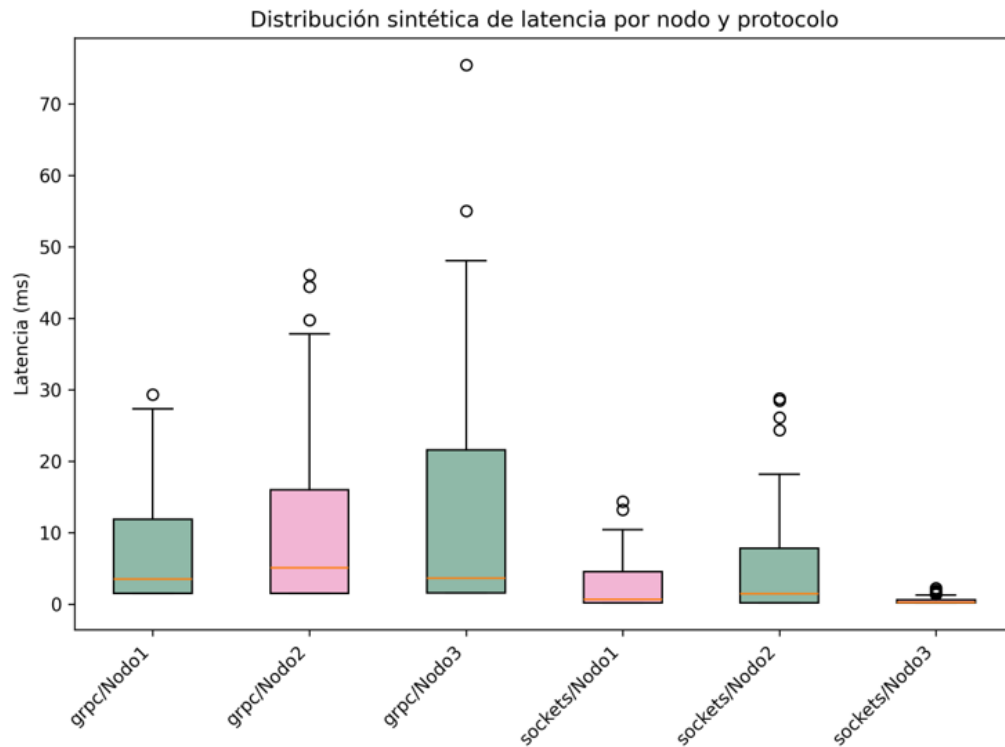


Figura 4.1: Diagrama de caja (*Boxplot*) comparativo de latencia a 300 DPI: Sockets TCP puros frente a gRPC en localhost.

4.3. Discusión y Respuestas a Preguntas Críticas

4.3.1. ¿Cuál de los enfoques evaluados tiene menor latencia?

Los resultados muestran que Sockets TCP tuvo menor latencia que gRPC. En promedio, los mensajes enviados mediante Sockets fueron más rápidos, alcanzando aproximadamente 0,57 ms, mientras que gRPC obtuvo alrededor de 3,05 ms.

Esto sucede porque Sockets TCP trabaja de forma más directa con la red y necesita menos procesos internos para enviar los datos. En cambio, gRPC utiliza tecnologías adicionales como HTTP/2 y Protocol Buffers, lo que agrega un pequeño tiempo extra de procesamiento.

4.3.2. ¿Cuál enfoque presenta menor variabilidad o dispersión?

Sockets TCP también presentó una variabilidad menor en los tiempos de respuesta. Esto significa que sus resultados fueron más estables y consistentes durante las pruebas.

Por otro lado, gRPC mostró una mayor dispersión en algunos casos, debido a los procesos internos que utiliza para administrar las conexiones y la comunicación entre cliente y servidor.

4.3.3. ¿Por qué se observan estas diferencias de rendimiento?

Las diferencias encontradas se deben principalmente a la forma en que cada tecnología maneja la comunicación.

Sockets TCP trabaja de manera más directa y simple, por lo que el envío de mensajes es más rápido. En cambio, gRPC incorpora herramientas adicionales que ayudan a estructurar mejor la comunicación y facilitan el desarrollo de aplicaciones modernas, aunque esto genera un pequeño aumento en la latencia.

Capítulo 5

Conclusiones y Bibliografía

5.1. Conclusión General

A través de esta práctica se logró comparar el comportamiento de Sockets TCP y gRPC dentro de un entorno distribuido utilizando tres nodos de comunicación. Además, se implementó el algoritmo de relojes lógicos de Lamport para mantener el orden de los eventos entre los procesos.

Los resultados obtenidos demostraron que Sockets TCP presentó menor latencia y tiempos de respuesta más rápidos durante las pruebas realizadas. Sin embargo, gRPC ofrece ventajas importantes relacionadas con la organización del código, la compatibilidad entre diferentes lenguajes y la facilidad para desarrollar aplicaciones distribuidas más escalables.

De manera general, ambas tecnologías son útiles dependiendo de las necesidades del sistema. Si se busca mayor velocidad y simplicidad, Sockets TCP resulta una buena opción. Por otro lado, si se requiere una arquitectura más estructurada y mantenible, gRPC puede ser más conveniente.

Bibliografía

- [1] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978. DOI: 10.1145/359545.359563
- [2] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Amsterdam, Netherlands: Maarten van Steen, 2017. ISBN: 978-1543057386
- [3] Google LLC, “gRPC: A high performance, open source, universal RPC framework,” *gRPC Documentation*, 2023. [En línea]. Disponible en: <https://grpc.io/docs/>
- [4] K. Indrasiri and D. Kuruppu, *gRPC: Up and Running: Building High-Performance APIs with gRPC*, 1st ed. Sebastopol, CA, USA: O’Reilly Media, 2020. ISBN: 978-1492058335
- [5] M. Belshe, R. Peon, and M. Thomson, “Hypertext Transfer Protocol Version 2 (HTTP/2),” *RFC Editor*, RFC 7540, May 2015. DOI: 10.17487/RFC7540
- [6] A. Chandra, R. Kumar, and S. Singh, “Performance comparison of Inter-Process Communication mechanisms in microservice architectures,” in *Proc. IEEE Int. Conf. Cloud Computing Technology and Science (CloudCom)*, Sydney, Australia, Dec. 2019, pp. 112–119. DOI: 10.1109/CloudCom.2019.00028